GEM 5 Exercise #2

114501583 陳冠晰 (Vic Chen)

Problem 1 (Sampling)

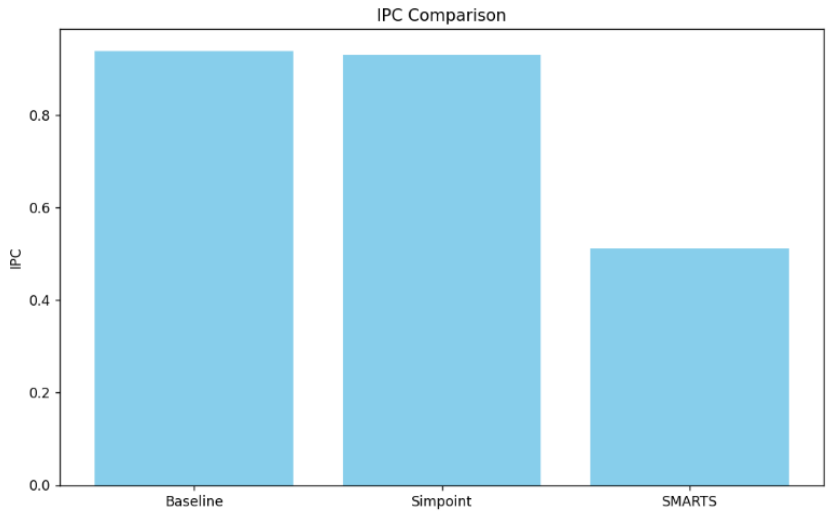
SimPoints result:

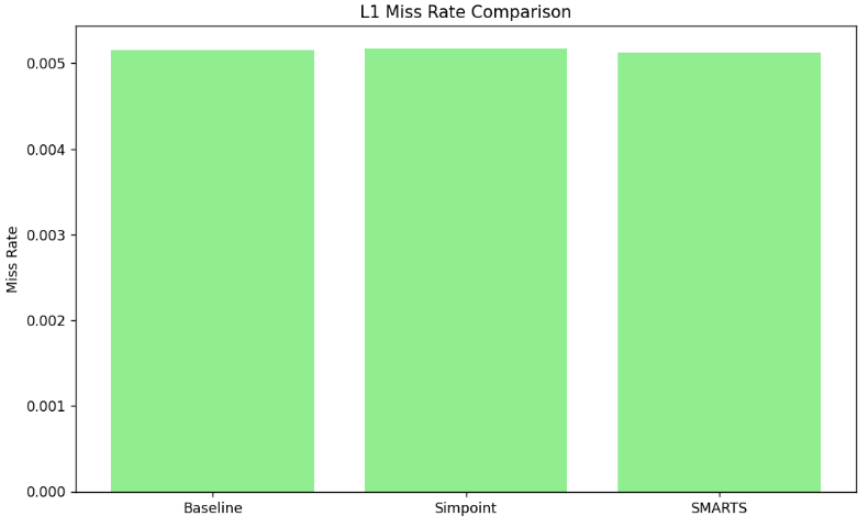
```
The sum is 57238500000
Full-detailed simulation Done
python3 simpoint_predict_ipc.py
predicted IPC: 0.937890181083
actual IPC: 0.939719
relative error: 0.19461338091492567%
root@EE6455-gem5:/workspaces/gem5-exercises/exercise2/p1#
```

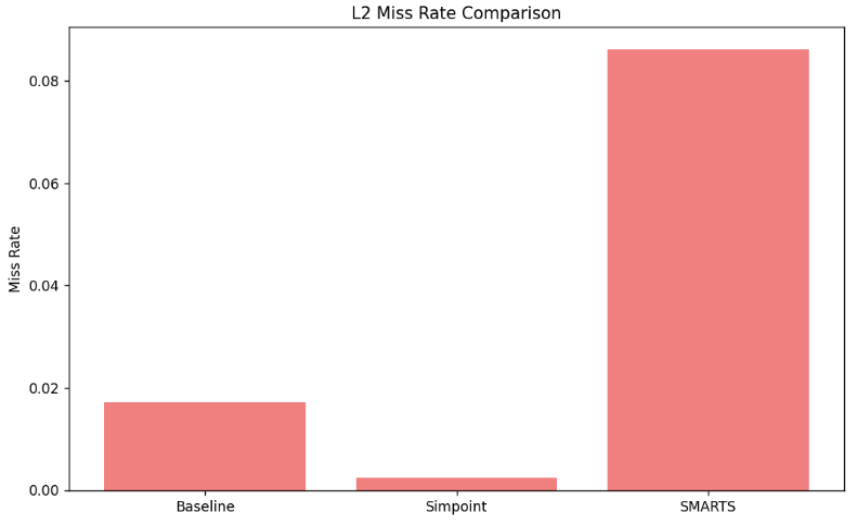
Smarts result:

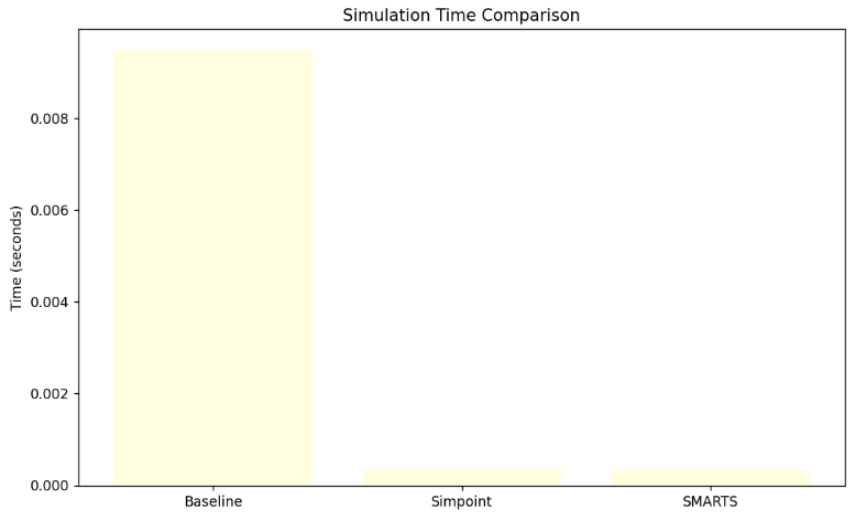
```
gem5 -re --outdir=log/smarts smarts.py
Redirecting stdout to log/smarts/simout.txt
Redirecting stderr to log/smarts/simerr.txt
python3 smarts_predict_ipc.py
Number of samples: 50
Predicted Overall IPC: 0.92440014
Actual Overall IPC: 0.939719
Relative Error: 1.6301532692219627%
root@EE6455-gem5:/workspaces/gem5-exercises/exercise2/p1#
```

Comparison:









The SMARTS sampling method effectively reduces simulation run time, but costs some

accuracy, particularly for metrics like L2 miss rate. While IPC and L1 miss rates remain fairly consistent across Baseline, Simpoint, and SMARTS, SMARTS shows a larger deviation in L2 miss rates. This indicates that the sampling parameters (intervals and warm-up periods) might not fully capture the system's behavior for more detailed metrics. The choice of sampling parameters (k, U, W) directly impacts the accuracy. A more refined sampling setup could improve the representativeness of SMARTS, especially for metrics that are sensitive to cache behavior, like L2 misses.

Problem 2 (Power Modeling)

Dynamic v.s. Static Power:

	config	cpu_dynamic_w	cpu_static_w	l3_dynamic_w	l3_static_w	total_dynamic_w	total_static_w	total_w
1	core_num2	8.008442	100.0	298.185217	100.0	306.193659	200.0	506.193659
2	core_num4	16.016884	100.0	298.185217	100.0	314.202101	200.0	514.2021010000001
3	cpu_freq_2GHz	8.008442	100.0	298.185217	100.0	306.193659	200.0	506.193659
4	cpu_freq_3GHz	8.012662	100.0	447.616726	100.0	455.629388	200.0	655.6293880000001
5	l3_size_2MB	8.008442	100.0	298.185217	100.0	306.193659	200.0	506.193659
6	l3_size_4MB	8.008442	100.0	298.185217	100.0	306.193659	200.0	506.193659

The analysis shows that dynamic power increases with both higher core counts and frequencies due to increased computational and memory access demands. However, the static power remains constant across configurations, which aligns with the general behavior of leakage currents, which do not depend on the system's activity level but rather on idle transistors.

A widely used real-world power estimation method for modern processors is the CMOS dynamic power equation:

$$P_{\text{dynamic}} = \alpha C V^2 f$$

- α is the activity factor, representing how frequently transistors switch.
- C is the capacitance of the circuit, which determines how much energy is needed to change the state of a transistor.
- V is the supply voltage, which directly impacts power consumption.
- f is the frequency of operation (clock speed).

This formula estimates the power consumed due to switching activity in the processor, which is the primary source of dynamic power. Static power is typically due to leakage currents and is given by the following formula:

$$P_{\text{static}} = I_{\text{leak}} V$$

Where I_{leak} represents the leakage current. Static power remains constant even when the system is idle, and it increases with the size of the device and its technology node.

To improve power efficiency, processors employ techniques like DVFS, which dynamically adjusts voltage and frequency based on workload demand, Power Gating, which disables power to unused components to reduce leakage, and MTCMOS, which uses different threshold voltages for optimizing speed and power. Additionally, Clock Gating reduces dynamic power by turning off the clock to inactive circuits, minimizing unnecessary switching activity. These methods collectively lower both dynamic and static power consumption.

Problem 3 (CHI Protocol)

Problem & Fixed:

```
warn: Base 10 memory/cache size 3GB will be cast to base 2 size 3GiB.
Global frequency set at 1000000000000 ticks per second
warn: board.workload.acpi_description_table_pointer.rsdt adopting orphan SimObject param 'entries'
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address rang
src/sim/kernel_workload.cc:46: info: kernel located at: workload/binaries/vmlinux-4.4.186
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does no
fatal: board.cache_hierarchy.ruby_system.sys_port_proxy.ruby_system without default or user set value
make: *** [makefile:29: chi_fs] Error 1
```

Running the original CHI example (hierarchy.py) will appear above error. I fixed this run-time error by assigning *ruby_system* to the per-core *RubySequencer* and to the *RubyPortProxy*. In gem5, all Ruby endpoints must hold a back-pointer to the owning *RubySystem* to register with the coherence network and share the common event context, missing this mandatory parameter can cause gem5 to abort during configuration.

CHI v.s. Classic:

p3_summary.csv

config	l2_overall_misses	l2_miss_rate	l2_miss_latency_mean	l2_ruby_m_demand_misses	chi_ReadUnique_total	chi_CleanUnique_total	chi_ReadShared_total	simSeconds	hostSeconds
classic_fs	73095.0	0.516076	59062.395061	nan	nan	nan	nan	0.00544	4.41
chi_fs	nan	nan	nan	85985.0	32565.0	6557.0	127492.0	0.009959	32.51

Discussion:

The Classic cache hierarchy uses a simpler mechanism where each cache accesses data directly through a shared interconnect without advanced protocols to handle coherence traffic across multiple cores. In contrast, the CHI protocol, used in the Ruby-

based cache hierarchy, is more sophisticated, introducing additional network hops and coherence lookups. But these extra operations come with overhead due to the complexity of maintaining cache consistency across multiple cores. The CHI supports advanced cache coherence protocols, which introduce more fine-grained control and state tracking but at the cost of increased communication traffic, as seen in the higher number of coherence transactions (ReadUnique, CleanUnique).

Following coherence events are crucial for maintaining data consistency in CHI protocol:

- ReadUnique: occurs when a cache requests data that is not present in any other cache and is obtained from memory.
- CleanUnique: occurs when a cache writes back data that was previously marked as modified in one of its cache lines.
- ReadShared: occurs when multiple caches access the same data, but it remains consistent across all involved caches.

The Classic overall misses represent the total cache misses including both L1 and L2 misses, which gives a general idea of how good the cache hierarchy is functioning. In the Ruby (CHI) hierarchy, the equivalent statistic is the demand misses (m), which counts the memory misses that are served through the Ruby network. The key difference lies in how these misses are handled: the Classic cache hierarchy may not track the finer granularity of coherence misses while the CHI can provide more detailed metrics about coherence traffic, giving a clearer concept of how cache coherence events contribute to overall system misses.